

Програмні засоби наукових обчислень

Лабораторна робота №2

Варіант 3

Студентки 2-го курсу
Спеціальності ПМ
Гирба М.С

<u>Завдання №1.....</u>	<u>3</u>
Завдання №2.....	4
Завдання №3.....	4
Завдання №4.....	5
Завдання №5.....	5

Завдання №1

Условие:

1. Даны 10 случайных точек $\vec{M}_i$ пространства $\mathbb{R}^2$ с целочисленными координатами.
 - a. Найти точку, наиболее удалённую от начала координат.
 - b. Отсортировать точки в порядке возрастания длин векторов $\vec{OM}_i$, где O - начало координат.
 - c. Описать функцию `v2normalize(x)`, которая по заданному вектору x возвращает нормированный вектор. Создать массив нормированных векторов $\vec{OM}'_i$ при помощи этой функции.
 - d. Профильтровать полученный массив, оставив только векторы с положительными координатами.

Код решения:

```
# -*- coding: utf-8 -*-

import numpy as np

points = np.random.randint(-10, 11, (10, 2))

distances = np.linalg.norm(points, axis=1)
farthest_point = points[np.argmax(distances)]

sorted_points = points[np.argsort(distances)]

def v2normalize(x):
    norm = np.linalg.norm(x)
    return x / norm if norm != 0 else x

norms = np.linalg.norm(sorted_points, axis=1, keepdims=True)
normalized_vectors = np.divide(sorted_points, norms, where=norms!=0)

positive_normalized_vectors =
normalized_vectors[np.all(normalized_vectors > 0, axis=1)]

print("Точки:\n", points)
print(f"Наиболее удаленная точка: {farthest_point}")
```

```
print("Сортировка в порядке возрастания длин векторов :\n",
sorted_points)
print("Массив нормированных векторов:\n",normalized_vectors)
print("Массив векторов с положительными координатами:\n",
positive_normalized_vectors)
```

Результат тестов:

```
Точки:
[[ 4  3]
 [-10  4]
 [ 0 -1]
 [ 4  2]
 [10  2]
 [-6 -1]
 [-8  8]
 [ 2 -7]
 [ 9 -6]
 [-10  3]]
Наиболее удаленная точка: [-8  8]
Сортировка в порядке возрастания длин векторов :
[[ 0 -1]
 [ 4  2]
 [ 4  3]
 [-6 -1]
 [ 2 -7]
 [10  2]
 [-10  3]
 [-10  4]
 [ 9 -6]
 [-8  8]]
Массив нормированных векторов:
[[ 0.          -1.         ]
 [ 0.89442719  0.4472136 ]
 [ 0.8         0.6        ]
 [-0.98639392 -0.16439899]
 [ 0.27472113 -0.96152395]
 [ 0.98058068  0.19611614]
 [-0.95782629  0.28734789]
 [-0.92847669  0.37139068]
 [ 0.83205029 -0.5547002 ]
 [-0.70710678  0.70710678]]
Массив векторов с положительными координатами:
[[0.89442719 0.4472136 ]
 [0.8        0.6        ]
 [0.98058068 0.19611614]]
```

Завдання №2

Условие: Сконструировать блочную матрицу (используя функции для заполнения стандартных матриц) и применить функции обработки данных и поэлементные операции для нахождения заданных величин.

$$A = \begin{pmatrix} 6 & 5 & 4 & 3 & 2 & 1 \\ -6 & -5 & -4 & -3 & -2 & -1 \\ 2 & 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 & 2 \\ 4 & 4 & 4 & 4 & 4 & 4 \\ 4 & 4 & 4 & 4 & 4 & 4 \end{pmatrix} \quad m = \min_{i,j=0,5} a_{ij}^3$$

Код решения:

```
import numpy as np

# Создание блоков для матрицы A
# block1 = np.array([[6, 5, 4, 3, 2, 1], [-6, -5, -4, -3, -2, -1]])
block1 = np.arange(6,0,-1)
block2 = np.arange(-6,0,1)
block3 = np.full((2, 6), 2)
block4 = np.full((2, 6), 4)

# Соединение блоков в одну матрицу A
A = np.vstack([block1, block2, block3, block4])

# Вычисление куба каждого элемента
A_cubed = A ** 3

# Найти минимальное значение среди элементов матрицы
min_value = np.min(A_cubed)

print("Матрица A:\n", A)
print("\nМатрица A, возведённая в куб:\n", A_cubed)
print("\nМинимальное значение среди элементов куба матрицы:", min_value)
```

Результат тестов:

```

Матрица A:
[[ 6  5  4  3  2  1]
 [-6 -5 -4 -3 -2 -1]
 [ 2  2  2  2  2  2]
 [ 2  2  2  2  2  2]
 [ 4  4  4  4  4  4]
 [ 4  4  4  4  4  4]]

Матрица A, возведённая в куб:
[[ 216 125  64  27  8  1]
 [-216 -125 -64 -27 -8 -1]
 [ 8  8  8  8  8  8]
 [ 8  8  8  8  8  8]
 [ 64 64 64 64 64 64]
 [ 64 64 64 64 64 64]]

Минимальное значение среди элементов куба матрицы: -216

```

Завдання №3

Условие: Написать функцию, проверяющую по критерию Сильвестра, является ли матрица положительно определённой. Проверить работу функции на следующих матрицах:

$$A = \begin{pmatrix} 14 & 6 & 3 \\ 6 & 9 & -4 \\ 3 & -4 & 9 \end{pmatrix} \quad B = \begin{pmatrix} -2 & 8 & 8 & 4 & -3 \\ 3 & 0 & -1 & 3 & 0 \\ 7 & -2 & -4 & 6 & 2 \\ 7 & -5 & 1 & -5 & -3 \\ -2 & -5 & 7 & 6 & -1 \end{pmatrix}$$

Код решения:

```

import numpy as np

def is_positive_definite(matrix):
    for k in range(1, matrix.shape[0] + 1):
        if np.linalg.det(matrix[:k, :k]) <= 0:
            return False
    return True

matrix_A = np.array([
    [14, 6, 3],
    [4, 9, -4],
    [3, -4, 9]
])

```

```

])

matrix_B = np.array([
    [2, 8, -8, 43, -3],
    [3, 0, -1, 3, 0],
    [7, -2, -4, 6, 2],
    [7, -5, 1, -5, -3],
    [-2, -5, 7, 6, -3]
])

is_A_positive_definite = is_positive_definite(matrix_A)
is_B_positive_definite = is_positive_definite(matrix_B)

print("Матрица A ", is_A_positive_definite)

print("Матрица B", is_B_positive_definite)

```

Результат тестов:

```

is_A_positive_definite True
is_B_positive_definite False

```

Завдання №4

Условие: При решении многих задач транспортной логистики возникает необходимость в построении матрицы расстояний между объектами.

Попробуем решить эту подзадачу в NumPy.

Сгенерировать 10 случайных точек на плоскости. Пусть, для определённости, эти точки попадают в квадрат, ограниченный прямыми $x = 0, y = 0, x = 10, y = 10$. Необходимо построить матрицу попарных расстояний между ними, если в качестве расстояния выбрана следующая метрика $\rho = (A, B) = \min\{|x_A - x_B|, |y_A - y_B|\}$.

На основе этой матрицы найти наиболее близкую (в указанной метрике) точку к складу, если считать, что координаты склада стоят в списке точек первыми.

Код решения:

```
import numpy as np

points = np.random.randint(0, 11, (10, 2))

dist_matrix = np.zeros((10, 10))
for i in range(10):
    for j in range(10):
        dist_matrix[i, j] = np.max(np.abs(points[i] - points[j]))

closest_point_index = np.argmin(dist_matrix[0, 1:]) + 1

print("Точки:\n", points)
print("Матрица расстояний:\n", dist_matrix)
print("Ближайшая точка к первой точке:", points[closest_point_index])
```

Результат тестов:

```
Точки:
[[10  9]
 [ 4  5]
 [ 6  3]
 [ 0  9]
 [ 4 10]
 [ 0  0]
 [ 7  5]
 [ 2  3]
 [ 4  1]
 [ 9  4]]
Матрица расстояний:
[[ 0.  6.  6. 10.  6. 10.  4.  8.  8.  5.]
 [ 6.  0.  2.  4.  5.  5.  3.  2.  4.  5.]
 [ 6.  2.  0.  6.  7.  6.  2.  4.  2.  3.]
 [10.  4.  6.  0.  4.  9.  7.  6.  8.  9.]
 [ 6.  5.  7.  4.  0. 10.  5.  7.  9.  6.]
 [10.  5.  6.  9. 10.  0.  7.  3.  4.  9.]
 [ 4.  3.  2.  7.  5.  7.  0.  5.  4.  2.]
 [ 8.  2.  4.  6.  7.  3.  5.  0.  2.  7.]
 [ 8.  4.  2.  8.  9.  4.  4.  2.  0.  5.]
 [ 5.  5.  3.  9.  6.  9.  2.  7.  5.  0.]]
Ближайшая точка к первой точке: [7 5]
```


Завдання №5

Условие: Написать функцию, заменяющую элемент матрицы с индексами 0, 0 суммой всех элементов матрицы. Функция считывает матрицу из файла input.csv и записывает результирующую матрицу в файл output.csv.

Код решения:

```
import numpy as np

# Определение функции для выполнения задачи 5
def replace_top_left_with_sum(input_file, output_file):
    """
    Функция считывает матрицу из файла input_file,
    заменяет элемент [0, 0] суммой всех элементов матрицы,
    и сохраняет изменённую матрицу в файл output_file.
    """
    # Чтение матрицы из файла
    matrix = np.loadtxt(input_file, delimiter=';')

    # Вычисление суммы всех элементов матрицы
    total_sum = np.sum(matrix)

    # Замена элемента [0, 0] на сумму
    matrix[0, 0] = total_sum

    # Запись изменённой матрицы в выходной файл
    np.savetxt(output_file, matrix, delimiter=';', fmt='%d')

# Создание примера input.csv с произвольной матрицей для демонстрации
input_matrix = np.random.randint(1, 10, (5, 5))
np.savetxt('LR2/input.csv', input_matrix, delimiter=';', fmt='%d')

# Применение функции к файлу input.csv и запись результата в output.csv
replace_top_left_with_sum('LR2/input.csv', 'LR2/output.csv')
```

Результат тестов:

input.csv U X		output.csv U X	
LR2 > input.csv > data		LR2 > output.csv > data	
1	2;3;5;8;5	1	143;3;5;8;5
2	9;9;9;3;9	2	9;9;9;3;9
3	7;8;7;2;1	3	7;8;7;2;1
4	1;9;3;6;4	4	1;9;3;6;4
5	8;4;9;4;8	5	8;4;9;4;8
6		6	